

# CHAPTER 3 语言设计思想和可综合性

---

## CHAPTER 3 语言设计思想和可综合性

引子

组合电路设计

例子：不同方式描述裁判表决器

具体器件设计

数字加法器

数据比较器

数据选择器MUX

38编码器

优先编码器

二化十BCD编码器

余三编码器

数字译码器

奇偶校验器

时序电路设计

触发器

计数器

移位寄存器

序列信号发生器

有限状态机

## 引子

描述一个模256的计数器：

#### 例4.1-1: 用Verilog HDL设计预256 (8bits) 计数器

##### (a) 可综合程序描述方式

```
module counter (count,clk,reset);
output count;
input clk,reset;
reg [7:0] count;
reg out;
always @(posedge clk)
    if (!reset) count<=0;
else if (count==8'b11111111) count<=0;
    else count<=count+1;
endmodule
```

##### (b) 常见的错误描述方式

```
module counter (count,clk,reset);
output count;
input reset,clk;
reg [7:0] count;
reg out;
integer i;
always @(posedge clk,reset)
begin
    if (!reset) count<=0;
    else
        for (i=0;i<=255;i=i+1)
            count<=count+1;
        end
end
endmodule
```

尽量少使用 **for** 进行描述！并且尽量使用边缘性进行触发。

#### 例4.1-2: 用Verilog HDL设计数字多路选择器

##### (a) 采用真值表形式的代码:

```
module MUX (out,data,sel);
output out;
input [3:0] data;
input [1:0] sel;
reg out;

always @(data or sel)
case (sel)
    2'b00 : out<=data[0];
    2'b01 : out<=data[1];
    2'b10 : out<=data[2];
    2'b11 : out<=data[3];
endcase
endmodule
```

##### (b) 采用逻辑表达式形式的代码:

```
module MUX (out,data,sel);
output out;
input [3:0] data;
input [1:0] sel;
wire w1,w2,w3,w4;

assign w1=(~sel[1])&(~sel[0])&data[0];
assign w2=(~sel[1])&sel[0]&data[1];
assign w3=sel[1]&(~sel[0])&data[2];
assign w4=sel[1]&sel[0]&data[3];
assign out=w1|w2|w3|w4;
endmodule
```

(c) 采用结构性描述的代码:

```
module MUX (out,data,sel);
output out;
input [3:0] data;
input [1:0] sel;
wire w1,w2,w3,w4;

nol      U1 (w1,sel[1]);
          U2 (w2,sel[0]);
and      U3 (w3,w1,w2,data[0]);
          U4 (w4,w1,sel[0],data[1]);
          U5 (w5,sel[1],w2,data[2]);
          U6 (w6,sel[1],sel[0],data[3]);
or       U7 (out,w3,w4,w5,w6);

endmodule
```

在现阶段,作为设计人员熟练掌握Verilog HDL程序设计的多样性和可综合性,是至关重要的。作为数字集成电路的基础,基本数字逻辑电路的设计是进行复杂电路的前提。本章通过对数字电路中基本逻辑电路的Verilog HDL程序设计进行讲述,掌握基本逻辑电路的可综合性设计,为具有特定功能的复杂电路的设计打下基础。

## 组合电路设计

组合电路的特点是:电路中任意时刻的稳态输出仅仅取决于该时刻的输入,而与电路原来的状态无关。

组合电路的设计需要从以下几个方面考虑:

所用的逻辑器件数目最少,器件的种类最少,且器件之间的连线最简单。这样的电路称"最小化"电路;

其次,为了满足速度要求,应使级数尽量少,以减少门电路的延迟;电路的功耗应尽可能的小,工作时稳定可靠。

描述组合逻辑电路有四利方式:结构描述、逻辑代数、真值表、抽象描述。

## 例子: 不同方式描述裁判表决器

首先,我们采用真值表描述方式

在Verilog HDL中,可以使用"case"语句对电路进行描述性设计,表4.2-1真值表设计代码如下:

```
module desingn (OUT,A,B,C);
output OUT;
input A,B,C;
reg OUT;

always @(A or B or C)
case ((A,B,C))
3'b000 : OUT<=0;
3'b001 : OUT<=0;
3'b010 : OUT<=0;
3'b100 : OUT<=0;
3'b011 : OUT<=1;
3'b101 : OUT<=1;
3'b110 : OUT<=1;
3'b111 : OUT<=1;
endcase

endmodule
```

采用卡诺图化简：使用结构性描述方式

根据逻辑函数表达式可以很方便写出采用逻辑代数描述方式的Verilog HDL程序。

```
module desingn (OUT,A,B,C);  
output OUT;  
input A,B,C;  
  
assign OUT=(A&B)|(B&C)|(A&C);  
  
endmodule
```

再者，我们使用抽象性描述方式：

#### 方法4：抽象描述方式

Verilog HDL还提供了采用抽象描述进行电路设计的方法，可以直接从电路功能出发，编写代码。例如判决器设计，将三个输入的判决相加，当判决成功时相加器之和大于1，即表示投票成功。

```
module desingn (OUT,A,B,C);  
output OUT;  
input A,B,C;  
wire [1:0] sum;  
reg OUT;  
  
assign sum=A+B+C;  
always @(sum)  
    if (sum>1)    OUT=1;  
    else          OUT=0;  
  
endmodule
```

首先使用加法器，再使用比较器即可

## 具体器件设计

### 数字加法器

Verilog HDL可以用不同的描述方式写出一位全加器，其综合电路是相同的，仅仅是描述风格不同。

(1) 利用连续赋值语句实现

```
module one_bit_fulladder ( SUM,
C_OUT, A, B, C_IN );
input  A, B, C_IN;
output SUM, C_OUT;

assign SUM=(A^B)^C_IN;
assign
C_OUT=(A&B)|((A^B)&C_IN);
//function of output

endmodule
```

(2) 利用行为描述实现

```
module one_bit_fulladder ( SUM,
C_OUT, A, B, C_IN );
output SUM, C_OUT;
input  A, B, C_IN;

assign {C_OUT,SUM}=A+B+C_IN;

endmodule
```

采用行为描述可以提高设计的效率，对于一个典型的多位加法器的行为描述设计，仅需改变代码中输入和输出信号的位宽即可，例如一个2输入8bits加法器，可以采用下面的Verilog HDL程序代码实现。

```
module eight_bit_fulladder ( SUM, C_OUT, A, B, C_IN );

output[7:0] SUM;
output C_OUT;
input [7:0] A, B;
input C_IN;

assign {C_OUT,SUM}=A+B+C_IN;

endmodule
```

利用超前加法器进行化简，降低时间延迟---->简化数学模型

## 4位超前进位加法器对应的Verilog HDL代码是:

```
module four_bits_fast_addder (sum_out,c_out,a,b,c_in);
input [3:0] a,b;           //the other of add number
input          c_in;           //carry in from before level
output [3:0] sum_out;       //the add of two input
output          c_out;           //carry out to next level
wire [4:0] g,p,c;           //wire between every c_out and c_in

assign c[0]=c_in;
assign p=a|b;
assign g=a&b;
assign c[1]=g[0]|(p[0]&c[0]);
assign c[2]=g[1]|(p[1]&(g[0]|(p[0]&c[0])));
assign c[3]=g[2]|(p[2]&(g[1]|(p[1]&(g[0]|(p[0]&c[0])))));
assign c[4]=g[3]|(p[3]&(g[2]|(p[2]&(g[1]|(p[1]&(g[0]|(p[0]&c[0])))))));
assign sum_out=p^c[3:0];
assign c_out=c[4];

endmodule
```

## 数据比较器

### 4位数据比较器真值表

```
module four_bits_comp1 ( F, A, B, C );
parameter comp_width=4;
output [2:0] F;
input [2:0] C;
input [comp_width-1:0] A;
input [comp_width-1:0] B;
reg [2:0] F;

always @(A or B or C)
    if (A>B)          F=3'b100;
    else if (A<B)     F=3'b001;
    else              F=C;

endmodule
```

## 数据选择器MUX

### 例4.2-5：8选1数据选择器

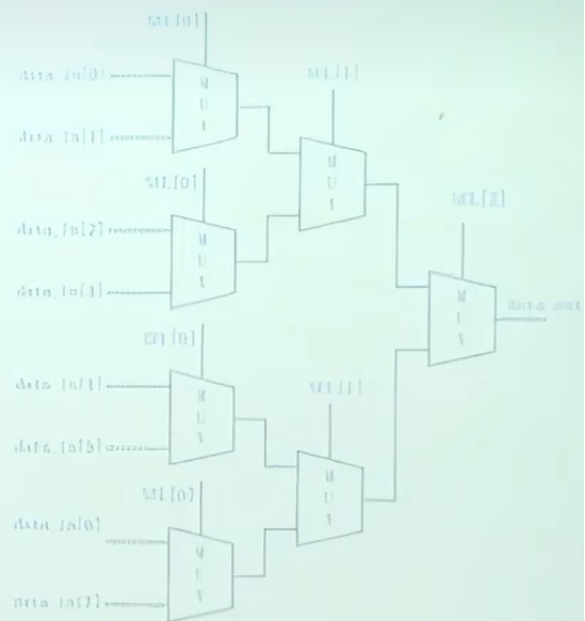
8选1数据选择器可以由多个2选1数据选择器设计。可以采用2选1数据选择器连接分成三级实现。

(1) 多个2选1数据选择器的结构级描述

```
module mux8to1_2( d_out, d_in, sel );
output    d_out;
input [7:0] d_in;
input [2:0] sel;
wire[3:0] w1;
wire[1:0] w2;

assign w1=sel[0]? (d_in[7],d_in[5],d_in[3],d_in[1]);
assign w2=sel[1]? (w1[3],w1[1]):(w1[2],w1[0]);
assign d_out=sel[2]?w2[1]:w2[0];

endmodule
```



### (2) 抽象描述方式

对于多路选择器的设计，可以采用“case”语句直接进行设计，在这种设计方式中，只需考虑选择信号列表，就可以实现功能更为复杂的数据选择器。

```
module mux8to1 (out,sel,data_in);
output out;
input [7:0] data_in;
input [3:0] sel;
reg out;
always @ (data_in or sel)
case (sel)
3'b000 : out<=data_in[0];
3'b001 : out<=data_in[1];
3'b010 : out<=data_in[2];
3'b011 : out<=data_in[3];
3'b100 : out<=data_in[4];
3'b101 : out<=data_in[5];
3'b110 : out<=data_in[6];
3'b111 : out<=data_in[7];
endcase
```

## 38编码器

### 3位二进制8线—3线编码器真值表

```
module code_8to3 (F,I);
output [2:0] F;
input [7:0] I;
reg [2:0] F;

always @ (I)
case (I)
8'b00000001: F=3'b000;
8'b00000010: F=3'b001;
8'b00000100: F=3'b010;
8'b00001000: F=3'b011;
8'b00010000: F=3'b100;
8'b00100000: F=3'b101;
8'b01000000: F=3'b110;
8'b10000000: F=3'b111;
default : F=3'bx;
endcase
endmodule
```

## 优先编码器

```
module mux8to3_p ( data_out, Ys, Yex, sel, data_in );
output [2:0] data_out;
output Ys, Yex;
input [7:0] data_in;
input sel;
reg [2:0] data_out;
reg Ys, Yex;
always @ (data_in or sel)
if (sel) (data_out, Ys, Yex) =(3'b111,1'b1,1'b1);
else
begin
casex (data_in)
8'b0??????? : (data_out, Ys, Yex) =(3'b000,1'b1,1'b0);
8'b10?????? : (data_out, Ys, Yex) =(3'b001,1'b1,1'b0);
8'b110????? : (data_out, Ys, Yex) =(3'b010,1'b1,1'b0);
8'b1110???? : (data_out, Ys, Yex) =(3'b011,1'b1,1'b0);
8'b11110??? : (data_out, Ys, Yex) =(3'b100,1'b1,1'b0);
8'b111110?? : (data_out, Ys, Yex) =(3'b101,1'b1,1'b0);
8'b1111110? : (data_out, Ys, Yex) =(3'b110,1'b1,1'b0);
8'b11111110 : (data_out, Ys, Yex) =(3'b111,1'b1,1'b0);
8'b11111111 : (data_out, Ys, Yex) =(3'b111,1'b0,1'b1);
endcase
end
endmodule
```

## 二化十BCD编码器

8421 BCD码编码表

| 十进制数        | D | C | B | A |
|-------------|---|---|---|---|
| 0 ( $Y_0$ ) | 0 | 0 | 0 | 0 |
| 1 ( $Y_1$ ) | 0 | 0 | 0 | 1 |
| 2 ( $Y_2$ ) | 0 | 0 | 1 | 0 |
| 3 ( $Y_3$ ) | 0 | 0 | 1 | 1 |
| 4 ( $Y_4$ ) | 0 | 1 | 0 | 0 |
| 5 ( $Y_5$ ) | 0 | 1 | 0 | 1 |
| 6 ( $Y_6$ ) | 0 | 1 | 1 | 0 |
| 7 ( $Y_7$ ) | 0 | 1 | 1 | 1 |
| 8 ( $Y_8$ ) | 1 | 0 | 0 | 0 |
| 9 ( $Y_9$ ) | 1 | 0 | 0 | 1 |

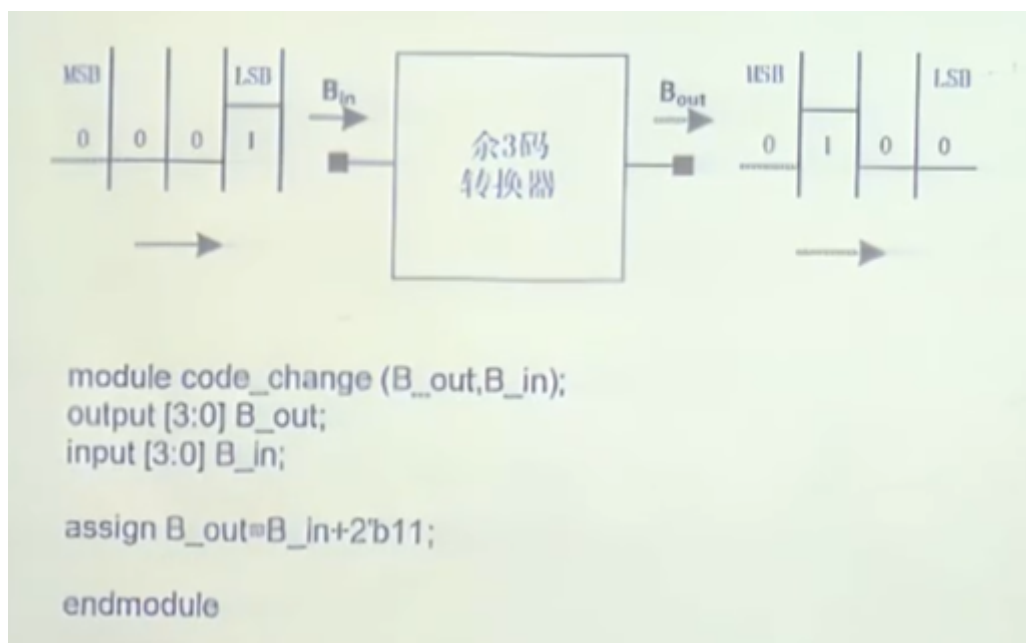
```

module BCD8421 ( data_out, data_in );
output [3:0] data_out;
input [8:0] data_in;
reg [3:0] data_out;

always @ (data_in)
case (data_in)
9'b000000000 : data_out=4'b0000;
9'b000000001 : data_out=4'b0001;
9'b000000010 : data_out=4'b0010;
9'b000000100 : data_out=4'b0011;
9'b000001000 : data_out=4'b0100;
9'b000010000 : data_out=4'b0101;
9'b000100000 : data_out=4'b0110;
9'b001000000 : data_out=4'b0111;
9'b010000000 : data_out=4'b1000;
9'b100000000 : data_out=4'b1001;
default : data_out=4'b0000;
endcase
endmodule

```

## 余三编码器

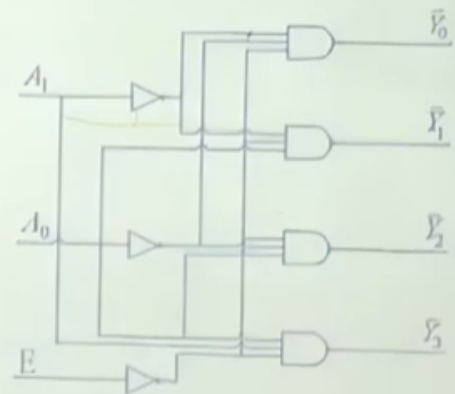


## 数字译码器

### • 4.2.5 数字译码器

译码是编码的逆过程，它将二进制代码所表示的信息翻译成相应的状态信息。实现译码功能的电路成为译码器。

图为2线—4线译码器的逻辑电路及逻辑符号。图中 $A_1, A_0$ 为地址输入端， $A_1$ 为高位。 $\bar{Y}_0, \bar{Y}_1, \bar{Y}_2, \bar{Y}_3$ 为状态信号输出端，非号表示低点平有效。E为使能端（或称选通控制端），低电平有效。当 $E=0$ 时，允许译码器工作， $\bar{Y}_0 \sim \bar{Y}_3$ 中只允许一个为有效电平输出；当 $E=1$ 时，禁制译码器工作，所有输出 $\sim$ 均为高电平。一般使能端有两个用途：一是可以引入选通脉冲，以抑制冒险脉冲的发生；二是可以用来扩展输入变量数（功能扩展）。



2线—4线译码器功能表

| E | $A_1$ | $A_0$ |   |   |   |   |
|---|-------|-------|---|---|---|---|
| 1 | X     | X     | 1 | 1 | 1 | 1 |
| 0 | 0     | 0     | 0 | 1 | 1 | 1 |
| 0 | 0     | 1     | 1 | 0 | 1 | 1 |
| 0 | 1     | 0     | 1 | 1 | 0 | 1 |
| 0 | 1     | 1     | 1 | 1 | 1 | 0 |

从表还可以看出，当 $E=0$ 时，2线—4线译码器的输出函数分别为：

$\bar{Y}_0 = \bar{A}_1 \bar{A}_0$ ,  $\bar{Y}_1 = \bar{A}_1 A_0$ ,  $\bar{Y}_2 = A_1 \bar{A}_0$ ,  $\bar{Y}_3 = A_1 A_0$ ，如果用表示i端的输出， $m_i$ 表示输入地址变量 $A_1, A_0$ 的一个最小项，则输出函数可写成

$\bar{Y}_i = \bar{E} m_i$  ( $i=0, 1, 2, 3$ ) 可见，译码器的每一个输出函数对应输入变量的一组取值，当使能端有效 ( $E=0$ ) 时，它正好是输入变量最小项的非。因此变量译码器也称最小项发生器。

很少采用高电位有效！

```

module decode_2to4 ( Y, E, A );
output [3:0] Y;
input [1:0] A;
input E;
assign Y[0] = ~(~E & ~A[1] & ~A[0]);
assign Y[1] = ~(~E & ~A[1] & A[0]);
assign Y[2] = ~(~E & A[1] & ~A[0]);
assign Y[3] = ~(~E & A[1] & A[0]);
endmodule

```

也可以采用抽象描述方式进行设计，Verilog HDL程序代码是：

```

module decode_2to4 ( Y, E, A );
output [3:0] Y;
input [1:0] A;
input E;
reg [3:0] Y;
always @(E or A)
case ((E,A))
3'b1?? : Y = 4'b0000;
3'b000 : Y = 4'b0001;
3'b001 : Y = 4'b0010;
3'b010 : Y = 4'b0100;
3'b011 : Y = 4'b1000;
default : Y = 4'b0000;
endcase
endmodule

```

## 奇偶校验器

使用异或电路判断

(a) 结构性描述代码：

```

module checker (Fod,Fev,b);
output Fod,Fev;
input [7:0] b;
wire w1,w2,w3,w4,w5,w6;

xor U1 (w1,b[0],b[1]);
xor U2 (w2,b[2],b[3]);
xor U3 (w3,b[4],b[5]);
xor U4 (w4,b[6],b[7]);
xor U5 (w5,w1,w2);
xor U6 (w6,w3,w4);
xor U7 (Fod,w5,w6);
not U8 (Fev,Fod);

endmodule

```

(b) 采用抽象性描述代码：

```

module checker (Fod,Fev,b);
output Fod,Fev;
input [7:0] b;

assign Fod = ^b;
assign Fev = ~Fod;

endmodule

```

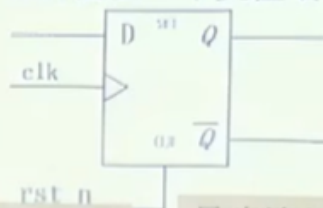
## 时序电路设计

利用有限状态机解决序列检测问题

## 触发器

### 例4.3-3: 带复位端D触发器

在D触发器的实际使用中,有时需要一个复位端(也称清零端)。



同步清零的D触发器

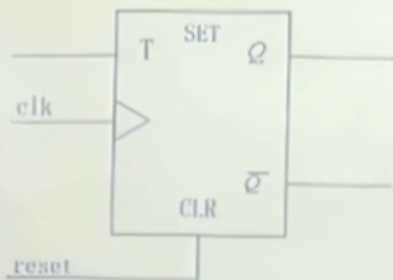
```
module DFF_rst ( q, clk, rst_n,
data_in );
output q;
input clk, rst_n, data_in;
reg q;
always @(posedge clk)
    if (!rst_n)      q<=0;
    else            q<=data_in;
endmodule
```

异步清零的D触发器

```
module DFF_srst ( q, clk, rst_n,
data_in, );
output q;
input clk, rst_n, data_in;
reg q;
always @(posedge clk or rst_n)
    if (!rst_n)      q<=0;
    else            q<=data_in;
endmodule
```

### 例4.3-5: T触发器

T触发器的逻辑符号如图4.3-10所示,其逻辑功能为:当时钟的有效边沿到来时,如果T=1,则触发器翻转;如果T=0,则触发器的状态保持不变。R为复位端,异步复位,低电平有效。

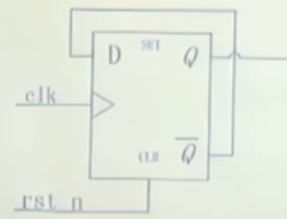


T触发器Verilog HDL程序代码是:

```
module TFF ( data_out, T, clk, rst_n );
output data_out;
input T, clk, rst_n;
reg data_out;
always @(posedge clk or rst_n)
    if (!rst_n)      data_out<=1'b0;
    else if (T)      data_out<=~data_out;
endmodule
```

## 计数器

例4.3-6: 2进制的计数器



```
module comp2bit ( Q, clk, rst_n);
output Q;
input clk, rst_n;
reg Q;
always @ (posedge clk or rst_n)
    if (!rst_n)
        Q<=1'b0;
    else
        Q<= ~Q;
endmodule
```

任意模的计数器:

例4.3-7:任意模值的计数器

任意模值M的计数器，第一步需要确定计数器所需要触发器个数。N个触发器对应了 $2^N$ 个状态。应有 $2^N > M$ 。任意模值计数器选取满足条件的最小N，N为计数器中触发器的个数。有两种方法实现：反馈清零法和反馈置数法。

下面是采用反馈清零法设计的模11计数器的Verilog HDL程序代码：

```
module comp_11 ( count, clk, rst );
output [3:0] count;
input clk,rst;
reg [3:0] count;
always @ (posedge clk)
    if (rst) count<=4'b0000;
    else
        if (count==4'b 1010)
            count<=4'b0000;
        else
            count<=count+1;
endmodule
```

反馈清零和反馈置数法进行计数

## 移位寄存器

```
module shiftregist1 (D,clk,rst_n);
parameter shiftregist_width=4;
output [shiftregist_width-1:0] D;
input clk,rst_n;
reg [shiftregist_width-1:0] D;

always @(posedge clk)
if (rst_n)
D<=4'b0000;
else
D<={D[shiftregist_width-2:0],D[shiftregist_width-1] };
endmodule
```

## 序列信号发生器

序列信号是数字电路系统中常用的功能单元，按照序列循环长度 $M$ 和触发器数目 $n$ 的关系一般可分为三种：

- (1) 最大循环长度序列码， $M=2^n$ 。
- (2) 最长线形序列码（ $m$ 序列码）， $M=2^n-1$

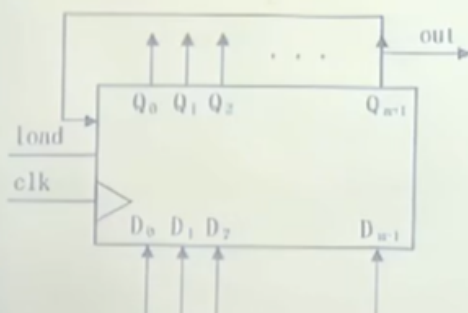
- (3) 任意循环长度序列码， $M<2^n$ 。

序列信号发生器是能够产生组或多组序列信号的时序电路，它可以由纯时序电路构成，也可以由包含时序和组合逻辑的混合电路构成。

例4.3-9:用Verilog HDL设计一个产生100111序列的信号发生器。

方法（1）：由移位寄存器构成

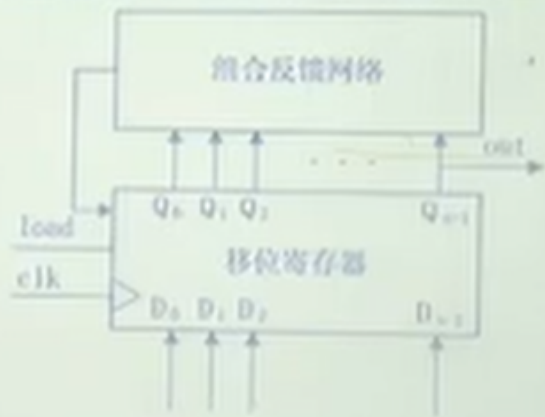
由于移位寄存器输入和输出信号之间没有组合电路，不需要经过组合逻辑的反馈运算，因此这种序列产生电路的工作频率很高。缺点是移位寄存器长度取决于序列长度，因此占用电路的面积很大。



```
module signal_maker(out,clk,load,D);
parameter M=6;
output out;
input clk,load;
input [M-1:0] D;
reg [M-1:0] Q;
initial Q=6'b100111;
always @(posedge clk)
if (load) Q<=D;
else
Q<={Q[M-2:0],Q[M-1]};
assign out=Q[M-1];
endmodule
```

方法(2): 由移位寄存器和组合逻辑电路构成

反馈移位型序列码发生器的结构框图如图所示, 它由移位寄存器和组合逻辑网络组成, 从移位寄存器的某一输出端可以得到周期性的序列码。



将给定的序列码按照移位规律每 $n$ 位一组, 划分为 $M$ 个状态。若 $M$ 个状态中出现重复现象, 则应增加移位寄存器位数。用 $n+1$ 位再重复上述过程, 直到划分为 $M$ 个独立状态为止。

(3) 根据 $M$ 个不同的状态列出移位寄存器的态序表和反馈函数表, 求出反馈函数 $F$ 的表达式。

(4) 检查自启动性能。

与上面的序列信号发生器相比, 各个寄存器的输出需要经过反馈网络, 然后才连接到移位寄存器的输入端。因此, 电路的速度必然下降, 但反馈网络的好处在于它可以节省寄存器。

对于"100111"序列的信号发生器。

首先, 确定所需移位寄存器的个数 $n$ 。因 $M=6$ 、故 $n \geq 3$ 。然后, 确定移位寄存器的六个独立状态。

按照规律每三位一组, 划分六个状态为100、001、011、111、111、110。其中状态111重复出现, 故取 $n=4$ , 并重新划分状态, 得到: 1001、0011、0111、1111、1110、1100。因此确定 $n=4$ 。

第三, 列态序表和反馈激励函数表, 求反馈函数 $F$ 的表达式。

首先列出态序表, 然后根据每一状态所需要的移位输入即反馈输入信号, 列出反馈激励函数表, 如下表所示。求得反馈激励函数:

$$F = \bar{Q}_3 + \bar{Q}_0 \bar{Q}_1 + \bar{Q}_2 Q_3$$

| $Q_3$ | $Q_2$ | $Q_1$ | $Q_0$ | $F$ |
|-------|-------|-------|-------|-----|
| 1     | 0     | 0     | 1     | 1   |
| 0     | 0     | 1     | 1     | 1   |
| 0     | 1     | 1     | 1     | 1   |
| 1     | 1     | 1     | 1     | 0   |
| 1     | 1     | 1     | 0     | 0   |
| 1     | 1     | 0     | 0     | 1   |

反馈移位型序列信号发生器的Verilog HDL程序代码是:

```
module signal_maker(out,clk,load,D);
    parameter M=4;
    output out;
    input clk,load;
    input [M-1:0] D;
    reg [M-1:0] Q;
    wire w1;

    always @(posedge clk)
        if (load)
            Q<=D;
        else
            Q<={Q[M-2:0],w1};
    assign w1=(~Q[3])&((~Q[1])&(~Q[0]))|(Q[3]&(~Q[2]));
    assign out=Q[M-1];
endmodule
```

反馈信号一旦错误就不能够工作

方法(3): 由计数器构成

计数型序列信号发生器和反馈型序列信号发生器大体相同,它们都是由时序电路和组合电路两部分构成。不同在于,反馈型序列信号发生器的时序状态由移位寄存器产生,输出取寄存器的最高位;而在计数型序列信号发生器中,采用计数器代替移位寄存器产生时序状态,输出由组合电路产生。

计数型的好处在于,计数器的状态设置与输出序列没有直接关系,不需要像上面一样,根据输出确定状态。只需要将反馈网络设计好就可以了。因此计数结构对于输出序列的更改比较方便,而且只要连接到不同的反馈网络,它可以同时产生多组序列码。



由真值表可画出输出Z的卡诺图，得到输出函数：

$$Z = Q_2 + \bar{Q}_2 \bar{Q}_0 + Q_2 Q_0$$

Verilog HDL程序代码是：

```
module signal_maker( OUT, clk, reset );
parameter M=3;
output OUT;
input clk, reset;
reg [M-1:0] counter;

always @(posedge clk)
if (reset) counter <= 3'b000;
    else counter <= counter + 1;
assign
OUT = counter[2] | ((~counter[1]) & (~counter[0])) | (counter[1] & counter[0]);
endmodule
```

错误不会传播，可靠性更高！

| N | F          |
|---|------------|
| 1 | 0          |
| 2 | 1, 0       |
| 3 | 1, 0       |
| 4 | 1, 0       |
| 5 | 2, 0       |
| 6 | 1, 0       |
| 7 | 1, 0       |
| 8 | 4, 3, 2, 0 |

通过不同的反馈网络，可以形成不同的移存型计数器。以m序列码为例，反馈函数如左表所示，表中的N是触发器的级数，F是反馈函数的列表。例如N=4，则反馈函数如下：

$$F = Q_1 \oplus Q_0$$

下面以N=4为例，在15位最长线性序列移存型计数器中，有一个由“0000”构成的死循环，为了打破死循环，可以修改式为

$$F = Q_1 \oplus Q_0 + \bar{Q}_3 \cdot \bar{Q}_2 \cdot \bar{Q}_1 \cdot \bar{Q}_0$$

根据N=4的最长线性序列移存型计数器的功能实现的伪随机码发生器Verilog HDL程序代码是：

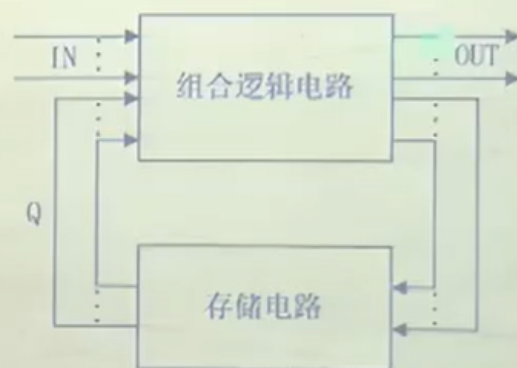
```
module signal15 ( out, clk, load_n, D_load );
output out;
input load_n, clk;
input [3:0] D_load;
reg [3:0] Q;
wire F;
always @(posedge clk)
    if (~load_n)      Q<=D_load;
    else
        Q<={Q[2:0],F};
assign F=(Q[1]^Q[0])|(~Q[3]&~Q[2]&~Q[1]&~Q[0]);
assign out=Q[3];
endmodule
```

## 有限状态机

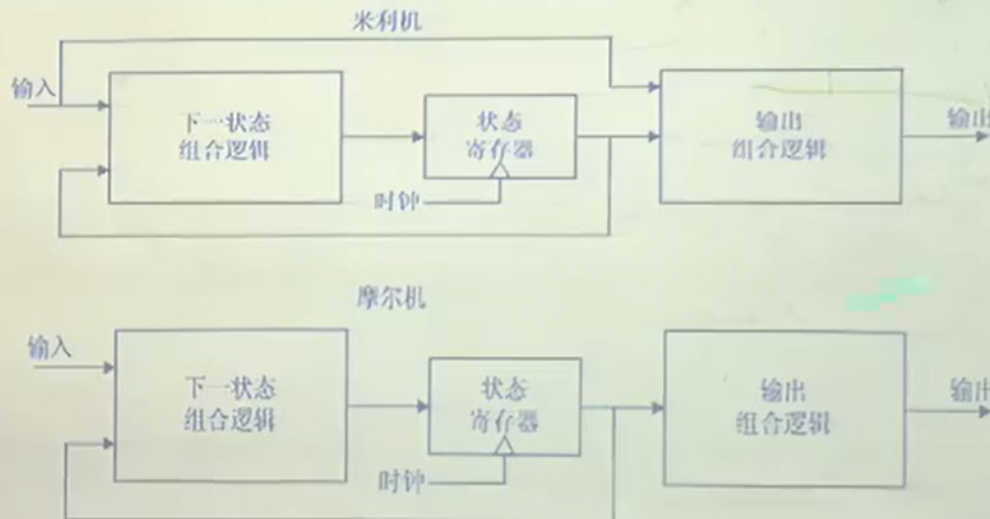
对于状态机，要尽量避免歧义的理解。比如序列检测中的歧义

有限状态机是时序电路的通用模型，任何时序电路都可以表示为有限状态机。有限状态机从本质上讲是由寄存器与组合逻辑构成的时序电路，各个状态之间的转移总是在时钟的触发下进行的，状态信息存储在寄存器中。因为状态的个数是有限的所以称为有限状态机。

同其它时序电路一样，有限状态机也是由两部分组成：存储电路和组合逻辑电路。存储电路，用来生成状态机的状态；组合逻辑电路，用来提供输出以及状态机跳转的条件。



根据输出信号的产生方式，有限状态机可以分为米利型（Mealy）和摩尔型（Moore）两类。Mealy型状态机的输出与当前状态和输入有关系，Moore型状态机的输出仅依赖当前状态而与输入无关。



摩尔型要更快一些！

使用格雷码进行编码，来达到避免暂态，但是某些情况下仍然是不行的

使用One Hot编码0001 0010 0100 1000但是位宽大，占用的资源很多

在Verilog HDL中，有限状态机的写法较多，常用的有两段式和三段式两种。

（1）状态机两段式描述方式：

```
//第一个进程，同步时序always模块，格式化描述次态寄存器迁移到现态寄存器
always @(posedge clk or negedge rst_n) //异步复位
if(!rst_n) current_state <= IDLE;
else current_state <= next_state; //注意，使用的是非阻塞赋值
//第二个进程，组合逻辑always模块，描述状态转移条件判断
always @(current_state和输入信号) //电平触发
begin
    next_state = x; //要初始化，使得系统复位后能进入正确的状态
    case(current_state)
    S1: if(...)
        next_state = S2; //阻塞赋值
        out1 <= 1'b1; //注意是非阻塞逻辑
    ...
    endcase
end
```

## (2) 状态机三段式描述方式:

```
//第一个进程，同步时序always模块，格式化描述次态寄存器迁移到现态寄存器
always @(posedge clk or negedge rst_n) //异步复位
if(!rst_n) current_state <= IDLE;
else
    current_state <= next_state; //注意，使用的是非阻塞赋值

//第二个进程，组合逻辑always模块，描述状态转移条件判断
always @(current_state和输入信号) //电平触发
begin
    next_state = x; //要初始化，使得系统复位后能进入正确的状态
    case(current_state)
    S1: if(...)
        next_state = S2; //阻塞赋值
```

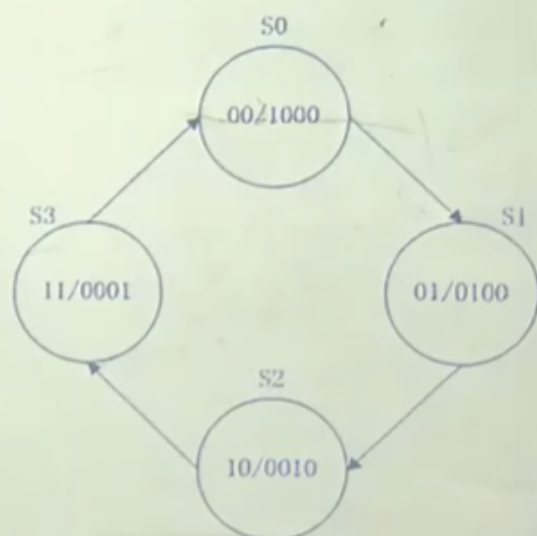
两段式和三段式是差不多的，不能产生latch

语法没有错误！但是电路错误的！

### 例4.4-1:用Verilog HDL设计顺序脉冲发生器

顺序脉冲发生器又称脉冲分配器，它将高电平的脉冲依次分配到不同输出上。保证在每个时钟内只有一路输出上是高电平脉冲，不同时钟上脉冲电平依次出现在所有输出。

以4位顺序脉冲发生器为例，它有四路输出 $W_0 W_1 W_2 W_3$ ，每路输出上高电平脉冲依次出现，输出在1000，0100，0010，0001之间循环。4位顺序脉冲发生器的状态转移图，如图所示。它由4个状态构成，每个状态中“1”的个数都是1个，表示每个时钟周期内只有一路输出端为高电平（脉冲），而且是轮流出现，因此生成顺序脉冲信号。



```

module state4 ( OUT, clk );
output [3:0] OUT;
input clk;
reg [3:0] OUT;
reg [1:0] STATE, next_STATE;
always @(STATE)
case (STATE)
2'b00:
begin
OUT<=4'b1000;
next_STATE<=2'b01;
end
2'b01:
begin
OUT<=4'b0100;
next_STATE<=2'b10;
end
2'b10:
begin
OUT<=4'b0010;
next_STATE<=2'b11;
end
2'b11:
begin
OUT<=4'b0001;
next_STATE<=2'b00;
end
endcase
always @(posedge clk)
STATE<=next_STATE;
endmodule

```

分为四种状态！

例4.4-2:设计一个卖报机，报纸价钱八角，纸币有1角，2角，5角，一元。该卖报机不考虑投币为大额面值等特殊情况。

右图是卖报机的状态转移图，图中S0~S7为状态机的8个状态，角标代表已投币的总合，如S0代表没有投币，S1代表已投入1角，依此类推。M代表输入，M1表示投入1角硬币，M2代表投入2角硬币，M5代表投入5角硬币，M10代表投入一元。

